

# AI-Powered Credit Risk Intelligence Platform

An end-to-end credit decisioning system on the Home Credit Default Risk dataset: it scores an applicant's probability of default, explains the score in plain English, states the policy rules behind it, and answers questions about the portfolio in natural language.

---

Dataset

**Home Credit Default Risk (Kaggle)**

Scale

**58,490,112 rows across 8 tables**

Labelled applications

**307,511 · 8.073% default**

Holdout ROC-AUC

**0.78989**

# A lender needs decisions that are fast, accurate, *and* defensible

---

In this dataset 8.073% of applicants default — roughly 1 in 12. Approving those loans costs written-off principal; refusing good applicants costs revenue. An underwriting desk therefore has six needs that a bare accuracy score does not meet.

- 1 Identify high-risk applicants early**  
Rank the book by risk before a human reads a file, so limited review capacity lands where losses actually are.
- 2 Automate the risk score itself**  
A calibrated probability per application, produced in one request, not a spreadsheet exercise per file.
- 3 Give a reason, not just a number**  
A declined applicant must be told which factors drove the decision, in language a non-analyst can act on.
- 4 Stand up to audit and regulation**  
Every figure traceable to a query or an artifact; the model's own limitations recorded rather than omitted.
- 5 Let non-technical staff query the data**  
A credit officer with a question about the portfolio should not need to file a ticket with an analyst.
- 6 Bridge the model into credit policy**  
A gradient-boosted ensemble cannot be written into a policy manual. A short list of measured rules can.

# Six modules, one running application

- 1
- Data pipeline**  
Kaggle download → DuckDB ingest of all 8 tables, with row counts verified against Kaggle's published figures on every build. 78.4s, 2.37 GB database.
- 2
- Exploratory analysis**  
Six data-quality checks and eight business insights, each backed by its own SQL query and rendered chart — no hand-typed numbers.
- 3
- Risk model**  
LightGBM over 205 features (122 raw + 83 engineered in SQL), isotonic-calibrated, with a four-way imbalance comparison behind the configuration.
- 4
- Explainability**  
SHAP TreeExplainer per applicant, turned into a 2–4 sentence narrative grounded strictly on the computed contributions.
- 5
- Business rules**  
A depth-4 surrogate tree distils the model into 16 IF-THEN rules, each with its measured population share and observed default rate.
- 6
- Talk-to-data chatbot**  
Natural-language question → generated SQL → AST validation → read-only execution → an answer written only from the returned rows.

All six are served by one FastAPI process that also hosts the UI — one image, one port, identical behaviour locally and in Docker.

CR

1 The portfolio

2 Score an applicant

3 Why this score

4 Policy rules

5 Ask the data

Ready, running on full data

### The portfolio

307,511 loan applications from the Home Credit Default Risk dataset. What the data holds, where it's incomplete, and eight findings worth a credit team's attention.

#### Dataset summary

| Table                 | Description                                                         | Rows       | Columns |
|-----------------------|---------------------------------------------------------------------|------------|---------|
| bureau_balance        | Monthly status history for each bureau credit.                      | 27,299,925 | 3       |
| installments_payments | Scheduled vs actual repayments - the repayment-behaviour signal.    | 13,605,401 | 8       |
| pos_cash_balance      | Monthly point-of-sale and cash-loan balance snapshots.              | 10,001,358 | 8       |
| credit_card_balance   | Monthly credit-card balance snapshots.                              | 3,840,312  | 23      |
| bureau                | Prior credits at other institutions, reported to the credit bureau. | 1,716,428  | 17      |
| previous_application  | Earlier Home Credit applications by the same clients.               | 1,670,214  | 37      |
| application_train     | One row per loan application, with the TARGET default label.        | 307,511    | 122     |
| application_test      | Unlabelled applications held out by the competition.                | 48,744     | 121     |
| column_descriptions   | Official column glossary. Feeds the chatbot's schema card.          | 219        | 5       |

#### Data quality

Six checks run directly against the live database — nothing here is asserted from memory.

action needed

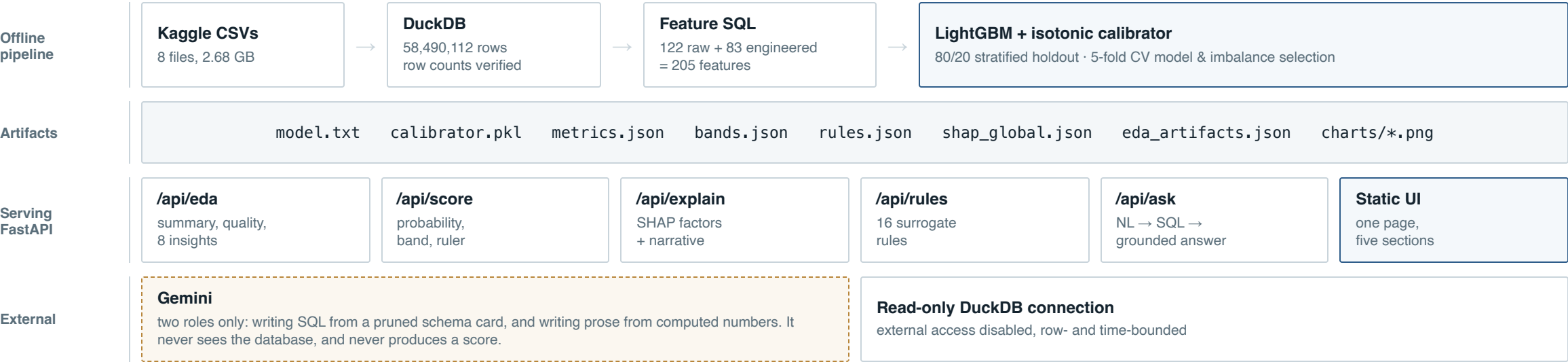
DAYS\_EMPLOYED uses 365243 (1000 years) as a sentinel for 'not employed' rather than a true value.  
count: 55,374 | pct: 18.01%

notable

CODE\_GENDER contains an 'XNA' (unknown) category.

The running application. Five sections in one working surface, matching the five parts of the problem — the portfolio, scoring, the explanation, the policy rules, and the chatbot.

# From Kaggle CSVs to a served, explained decision



The model is **never** in the LLM's path. Scores, SHAP values, bands and rules are all computed numerically; the LLM only renders them into sentences.

# Eight tables, 58.5 million rows, one label

| Table                    | Rows           | Cols       | What it holds                             |
|--------------------------|----------------|------------|-------------------------------------------|
| <b>application_train</b> | <b>307,511</b> | <b>122</b> | <b>Applications with the TARGET label</b> |
| application_test         | 48,744         | 121        | Unlabelled held-out applications          |
| bureau                   | 1,716,428      | 17         | Prior credits at other institutions       |
| bureau_balance           | 27,299,925     | 3          | Monthly status on those credits           |
| previous_application     | 1,670,214      | 37         | Earlier applications to this lender       |
| installments_payments    | 13,605,401     | 8          | Scheduled vs. actual repayments           |
| credit_card_balance      | 3,840,312      | 23         | Monthly credit-card balances              |
| pos_cash_balance         | 10,001,358     | 8          | Monthly POS / cash balances               |

58,489,893 data rows plus a 219-row Kaggle column glossary = **58,490,112**. Every count matches Kaggle's published figures, re-verified on each build. Only **application\_train** carries a label; the other seven contribute per-client aggregates as engineered features.

**307,511**

labelled applications

**8.073%**

default rate — roughly 1 applicant in 12

**11.4 : 1**

class imbalance, the central modelling constraint

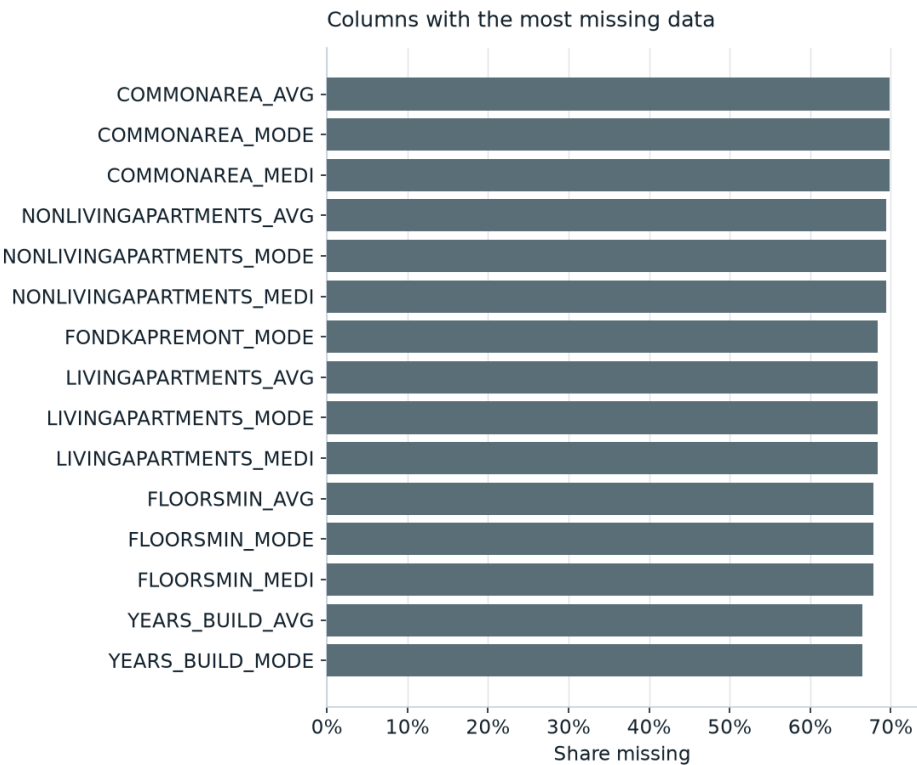
**205**

features: 122 raw application columns plus 83 engineered in SQL

At 11.4:1, **accuracy is meaningless** — predicting "no default" for everyone scores about 92%. Every model decision in this project is made on **PR-AUC** instead.

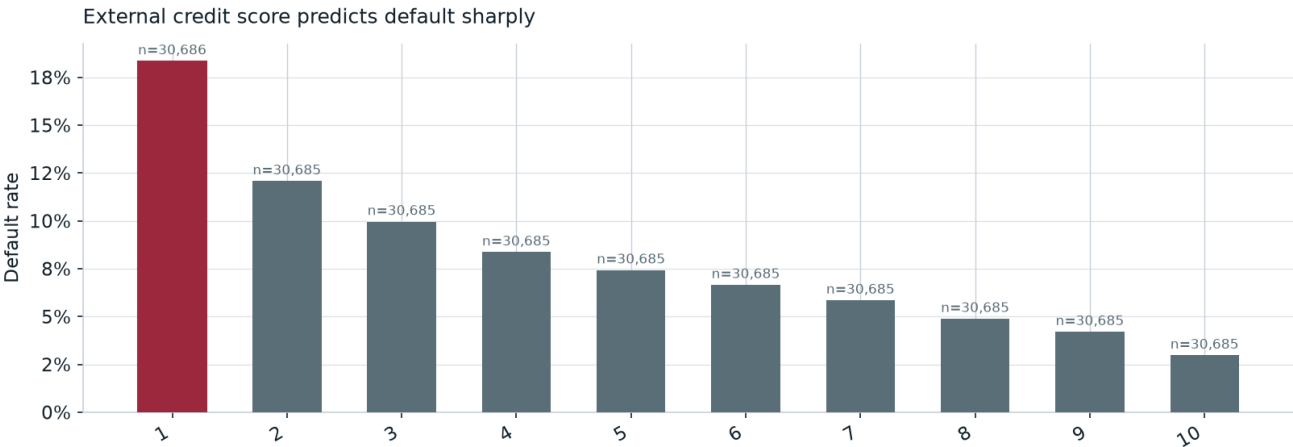
# Six checks, run against the live database

- **DAYS\_EMPLOYED sentinel.** 55,374 rows (18.0%) carry 365243 — 1,000 years — meaning "not employed", not a tenure. Left alone it would dominate any split on employment length. Mapped to NULL, with the fact kept as FLAG\_NOT\_EMPLOYED.
- **CODE\_GENDER = 'XNA'.** 4 rows. Negligible in volume, but an unhandled category would silently become its own one-hot column.
- **AMT\_INCOME\_TOTAL outlier.** Maximum 117,000,000 against a median of 147,150 — a 795x ratio, one applicant, almost certainly a data-entry artefact. Handled by tree splits, which are scale-immune, rather than by deleting the row.
- **DAYS\_\* sign convention.** All 4 non-DAYS\_EMPLOYED date-offset columns checked for unexpected positive values. none found
- **Building-statistics missingness.** The 42-column housing block is 48.8%–69.9% missing. Not random — it is absent for applicants whose housing type has no building record. Missingness is itself informative, so it is left as NaN for LightGBM rather than imputed.
- **Class imbalance.** 8.07% positive, 11.4:1 — addressed with the documented four-way comparison on slide 10.



Top columns by missingness. Every one belongs to the same housing-detail block — which is why the fix is a modelling decision (native NaN handling), not imputation.

# External credit score is the strongest single signal in the dataset



18.4%

default rate, bottom EXT\_SOURCE\_2 decile

3.0%

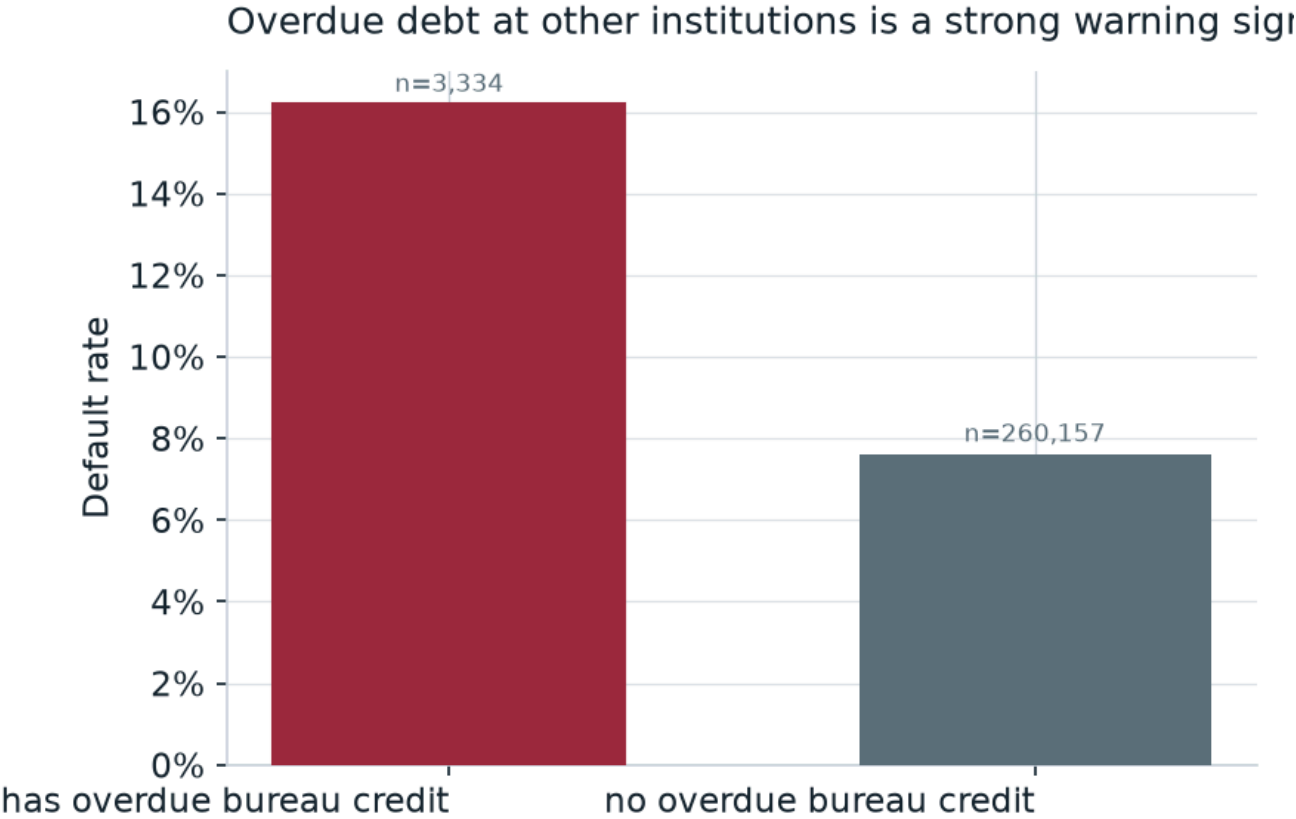
default rate, top decile

6.2x

risk gap across deciles — monotonic, with no dip or reversal anywhere in the curve

**So what:** EXT\_SOURCE\_2 and its siblings should anchor both the model and any manual review process. Few raw application fields separate risk this cleanly on their own — and the SHAP ranking on slide 12 confirms the model reached the same conclusion independently.

# Overdue debt elsewhere roughly doubles the default rate



**16.2%**

default rate — any overdue balance on a bureau-reported credit

**7.6%**

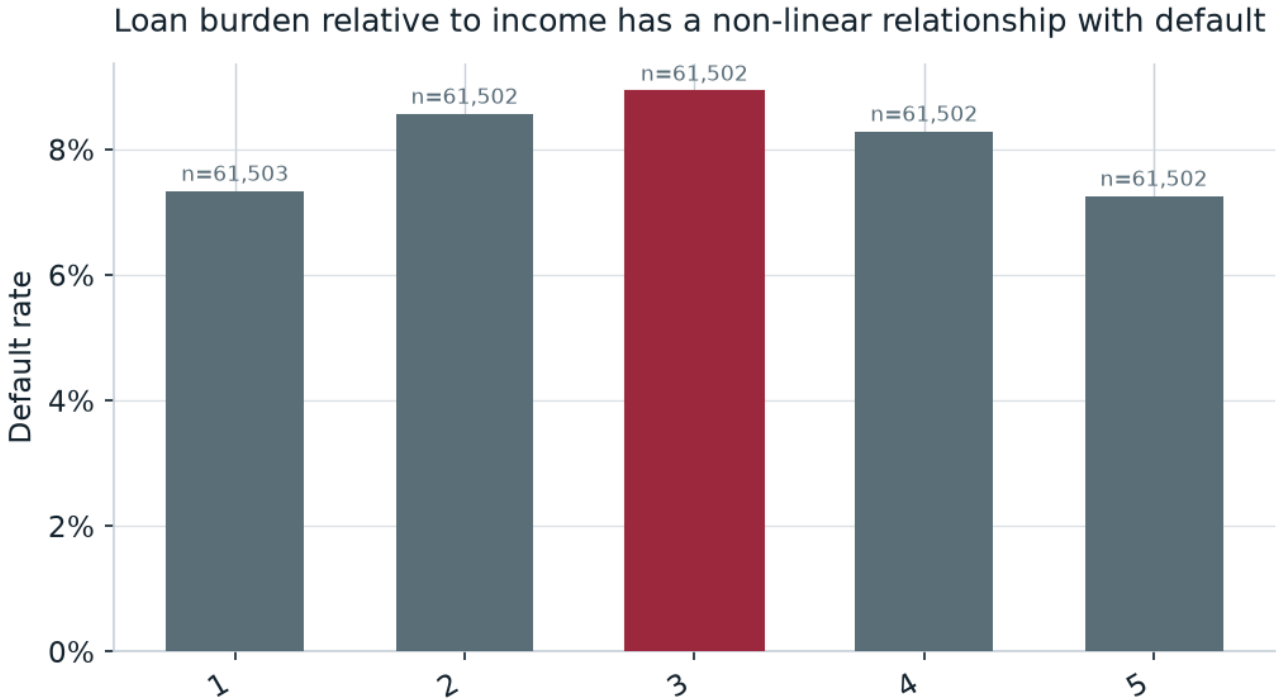
default rate — no overdue bureau credit

**2.1X**

relative risk, against a portfolio base rate of 8.07%

**So what:** bureau data surfaces problems that are invisible to the application form alone — obligations already in trouble at another lender. This is the clearest argument in the analysis for joining the child tables at all rather than modelling the application form on its own.

# Loan burden and default are *not* a straight line



8.9%

default rate,  
**middle**  
quintile — average credit at 3.3x income

7.3%

default rate,  
**heaviest**  
quintile — average credit at 8.2x income

**So what:** the most heavily indebted applicants are *not* the riskiest ones. A naive "flag high credit-to-income ratio" policy rule would miss the actual riskiest segment entirely.

This finding also caught a real bug: the insight function originally assumed the relationship was monotonic and read the riskiest quintile off the end of the sorted list. Both this and a reversed EXT\_SOURCE lift calculation are now covered by regression tests with known, checkable answers.

# Model choice, and the imbalance experiment

**Why LightGBM.** At ~307k rows and 205 mixed numeric/categorical features, gradient-boosted trees are the established strong baseline for tabular data at this scale — no GPU, minutes to train. LightGBM specifically for native NaN handling (the missingness here is informative, not random) and native categorical support, which avoids a 58-level one-hot expansion of ORGANIZATION\_TYPE.

| Model (identical feature set)              | Mean PR-AUC, 5-fold CV |
|--------------------------------------------|------------------------|
| Logistic Regression (transparent baseline) | 0.2518                 |
| LightGBM (shipped)                         | 0.2790                 |

**+10.8% relative** over the linear reference — the honest answer to "how much does model choice buy," isolated from the feature-engineering contribution.

Four imbalance strategies, same CV splits, selected on PR-AUC:

| Strategy                 | PR-AUC | ROC-AUC | Brier  |
|--------------------------|--------|---------|--------|
| None (shipped)           | 0.2790 | 0.7859  | 0.0660 |
| Random undersampling     | 0.2725 | 0.7818  | 0.1884 |
| SMOTE                    | 0.2492 | 0.7688  | 0.0675 |
| scale_pos_weight (≈11.4) | 0.2000 | 0.7330  | 0.0733 |

**None of the three standard imbalance fixes beat training on the data as-is.**  
scale\_pos\_weight — the one most commonly reached for — is the *worst* performer here. Undersampling's Brier score of 0.1884, nearly 3x worse, shows why: discarding 90% of the majority class destroys the model's sense of how rare a default actually is, so its probabilities become badly overconfident.

The shipped model therefore uses **no explicit imbalance handling**. That is a measured result, not an assumption — and it runs against the common expectation that imbalance always needs correcting.

# What the model is worth to an underwriting desk

|                                       |                                                                            |                                            |                                                                     |
|---------------------------------------|----------------------------------------------------------------------------|--------------------------------------------|---------------------------------------------------------------------|
| <div>0.78989</div> <div>ROC-AUC</div> | <div>0.27939</div> <div>PR-AUC — the metric every choice was made on</div> | <div>0.44141</div> <div>KS statistic</div> | <div>0.06537</div> <div>Brier score, calibrated (0.06560 raw)</div> |
|---------------------------------------|----------------------------------------------------------------------------|--------------------------------------------|---------------------------------------------------------------------|

**Risk bands**, cut from the calibrated holdout score distribution — not from arbitrary probability thresholds, since a "0.3 cutoff" means nothing on its own at an 8% base rate.

|           |            |              |                  |            |
|-----------|------------|--------------|------------------|------------|
| Low — 50% |            | Medium — 40% |                  | High — 10% |
| Band      | Population | Score ≥      | Observed default | Lift       |
| High      | 10%        | 0.1878       | 29.9%            | 3.7x       |
| Medium    | 40%        | 0.0449       | 9.7%             | 1.2x       |
| Low       | 50%        | —            | 2.4%             | 0.3x       |

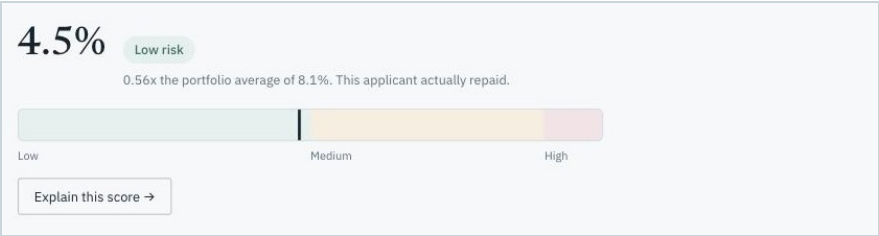
**Lift by decile** — the table a credit officer reads.

| Decile (riskiest first) | Default rate | Lift  | Cum. defaulters caught |
|-------------------------|--------------|-------|------------------------|
| 1                       | 29.9%        | 3.70x | 37.0%                  |
| 2                       | 16.6%        | 2.06x | 57.6%                  |
| 3                       | 10.3%        | 1.28x | 70.4%                  |
| 4                       | 7.1%         | 0.88x | 79.3%                  |
| 5                       | 4.7%         | 0.58x | 85.1%                  |

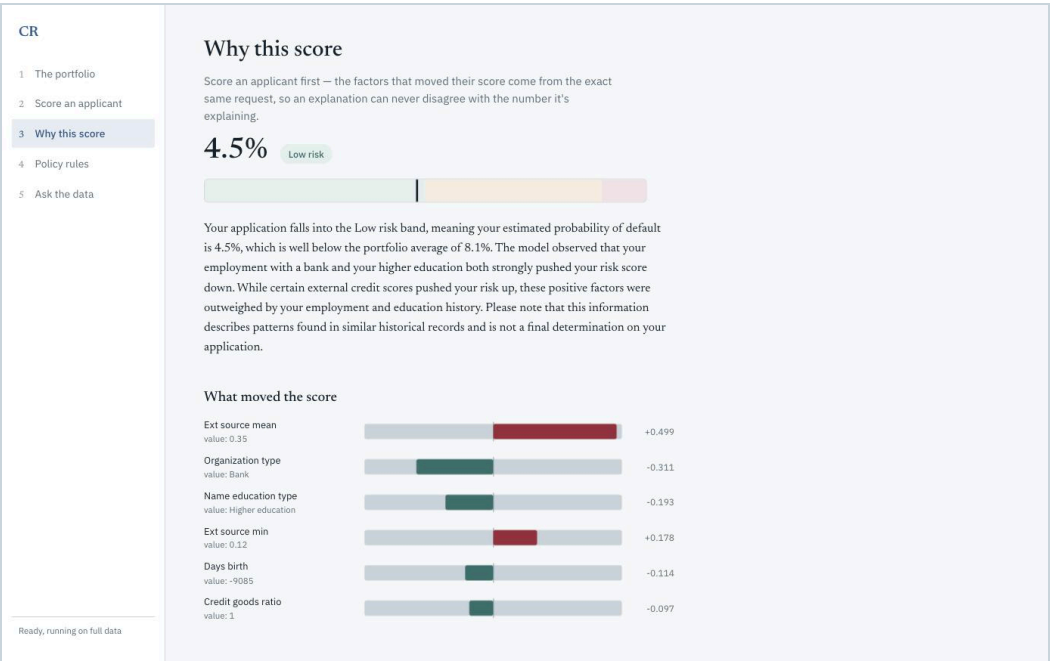
**Reviewing the riskiest 30% of applicants catches 70.4% of all eventual defaulters.** That is the concrete number behind "prioritise manual review by risk score" — a 3.3x concentration of review effort.

Operating threshold **0.0935**, set by minimising expected cost under a stated 10:1 false-negative-to-false-positive cost assumption — not by a 0.5 cutoff or by F1, neither of which is meaningful for a lender. Precision 20.0%, recall 67.3% at that threshold.

# Every score comes with its reasons — including the uncomfortable one



Step 1 — a real applicant scored at 4.5%, placed on the portfolio's own score distribution rather than shown as a bare number.



Step 2 — the same applicant's SHAP narrative and push/pull factor chart, computed from the same request.

SHAP's TreeExplainer runs directly on the LightGBM booster. The factors and the score come from the *same* request, so an explanation can never disagree with the number it explains.

**One distinction, stated because it is easy to get backwards:** SHAP explains the model's raw log-odds output, not the calibrated probability shown to the user — calibration is a monotone remap fitted afterwards, with no per-feature decomposition. Contributions are therefore always presented directionally ("pushed the risk up / down"), never as an exact slice of the displayed percentage.

**Top global features** by mean ISHAPI:

|   |                    |       |
|---|--------------------|-------|
| 1 | EXT_SOURCE_MEAN    | 0.404 |
| 2 | CODE_GENDER        | 0.104 |
| 3 | ORGANIZATION_TYPE  | 0.099 |
| 4 | CREDIT_GOODS_RATIO | 0.093 |

**CODE\_GENDER is the model's #2 global feature** — ahead of income, credit amount and employment history. Using a protected attribute as a predictor raises real fair-lending concerns. This is **recorded as a limitation in the model card, not hidden**: a production deployment would need to exclude protected attributes or apply an explicit fairness constraint and audit first.

# The model, distilled into a checklist a policy manual can hold

A depth-4 decision tree is fitted to approximate the *calibrated score* (not the label), then each leaf is reported as an IF-THEN rule. **16 rules** were extracted, from 36.3% down to 2.6% observed default rate. Every rate below is measured directly against TARGET for the applicants actually in that leaf — not read off the tree's own prediction.

- 1

IF **ext-source mean  $\leq$  0.253** AND **ext-source min  $\leq$  0.121** AND **credit-to-goods ratio  $>$  1.16**  
→ 1.5% of applicants (4,554) · observed default rate **36.3%** · **4.5x** base rate
- 3

IF **ext-source mean  $\leq$  0.395** AND  **$>$  0.253** AND **previous-refusal rate  $>$  0.311** AND **credit-to-goods ratio  $>$  1.16**  
→ 1.3% of applicants (4,046) · observed default rate **23.1%** · **2.87x** base rate
- 4

IF **ext-source mean  $\leq$  0.253** AND **ext-source min  $>$  0.121** AND **years employed  $\leq$  11.6**  
→ 2.1% of applicants (6,400) · observed default rate **22.9%** · **2.84x** base rate

**Fidelity, reported rather than assumed.** The surrogate reaches  $R^2 = 0.4885$  against the calibrated score and ROC-AUC **0.7207** against the actual label, versus the full model's **0.8766** on the same data. It captures real, useful structure but is **meaningfully simpler than the model it approximates** — a readable four-question checklist, not a substitute for the model in a decision that matters.

CR

1 The portfolio

2 Score an applicant

3 Why this score

4 Policy rules

5 Ask the data

Ready, running on full data

Policy rules

A simplified, four-question decision tree fit to approximate the model's score — the kind of checklist a credit policy document could contain. Every number below is measured directly against real outcomes for applicants matching that rule, not read off the tree's own prediction.

This simplified tree explains 49% of the full model's score variation ( $R^2$ ), and ranks applicants with 72% ROC-AUC against the full model's 88% — a useful, readable summary, not a replacement for the model.

|   |                                                                                                                                  |                                                  |
|---|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| 1 | Ext source mean $\leq$ 0.395 and Ext source mean $\leq$ 0.253 and Ext source min $\leq$ 0.121 and Credit goods ratio $>$ 1.16    | 36.3%<br>4.5x the base rate, 1.5% of applicants  |
| 2 | Ext source mean $\leq$ 0.395 and Ext source mean $\leq$ 0.253 and Ext source min $\leq$ 0.121 and Credit goods ratio $\leq$ 1.16 | 25.9%<br>3.21x the base rate, 1.8% of applicants |
| 3 | Ext source mean $\leq$ 0.395 and Ext source mean $>$ 0.253 and Prev refusal rate $>$ 0.311 and Credit goods ratio $>$ 1.16       | 23.1%<br>2.87x the base rate, 1.3% of applicants |
| 4 | Ext source mean $\leq$ 0.395 and Ext source mean $\leq$ 0.253 and Ext source min $>$ 0.121 and Years employed $\leq$ 11.6        | 22.9%<br>2.84x the base rate, 2.1% of applicants |
| 5 | Ext source mean $\leq$ 0.395 and Ext source mean $>$ 0.253 and Prev refusal rate $\leq$ 0.311 and Credit goods ratio $>$ 1.21    | 18.2%<br>2.26x the base rate, 3.1% of applicants |
| 6 | Ext source mean $\leq$ 0.395 and Ext source mean $>$ 0.253 and Prev refusal rate $>$ 0.311 and Credit goods ratio $\leq$ 1.16    | 17.6%<br>2.18x the base rate, 1.8% of applicants |
| 7 | Ext source mean $>$ 0.395 and Ext source mean $\leq$ 0.517 and Prev granted ratio $>$ 1.06 and Cc drawings count mean $>$ 1.14   | 16.6%<br>2.06x the base rate, 0.8% of applicants |
| 8 | Ext source mean $\leq$ 0.395 and Ext source mean $>$ 0.253 and Prev refusal rate $\leq$ 0.311 and Credit goods ratio $\leq$ 1.21 | 12.8%<br>1.58x the base rate, 9.4% of applicants |

All 16 rules in the running application, each with its measured population share and lift, and the fidelity caveat stated at the top of the list rather than in a footnote.

# A plain-English question, an auditable answer

CR

1 The portfolio

2 Score an applicant

3 Why this score

4 Policy rules

5 Ask the data

Ask the data

Ask a plain-English question about the portfolio. The SQL that ran is always shown next to the answer, so nothing here is asserted without a query behind it.

e.g. How does default rate differ by gender?

Ask

Try:

What is the overall default rate?

Which education level defaults most?

Do clients with overdue bureau credit default more?

Which education level defaults most?

Clients with a lower secondary education have the highest default rate at 10.9%. This compares to a low of 1.8% for those with an academic degree, and 8.9% for the largest group, which holds a secondary or secondary special education.

SELECT  
NAME\_EDUCATION\_TYPE,  
COUNT(\*) AS applications,  
SUM(TARGET) AS defaults,  
AVG(TARGET) AS default\_rate  
FROM application\_train  
GROUP BY  
NAME\_EDUCATION\_TYPE  
ORDER BY  
default\_rate DESC  
LIMIT 500

| NAME_EDUCATION_TYPE           | applications | defaults | default_rate         |
|-------------------------------|--------------|----------|----------------------|
| Lower secondary               | 3816         | 417      | 0.10927672955974843  |
| Secondary / secondary special | 218391       | 19524    | 0.08939928843221562  |
| Incomplete higher             | 10277        | 872      | 0.08484966429891992  |
| Higher education              | 74863        | 4009     | 0.05355115344028425  |
| Academic degree               | 164          | 3        | 0.018292682926829267 |

5 row(s) 2100 tokens 2656ms

Ready, running on full data

A real round-trip against the live Gemini API: the question, the answer, the exact SQL that ran, the returned rows, and the token and latency cost — all shown together.

## The pipeline

### 1. Schema card, pruned per question

Two tiers — every table's purpose and join keys always; full column detail only for tables the question plausibly touches. Deterministic lexical selection, no embeddings.

### 2. Gemini writes SQL

Sees only columns that exist. Given an explicit way to refuse, so an unanswerable question need not be answered with an invented one.

### 3. sqlglot AST validation

Parsed, not regex-matched: one statement, read-only, no filesystem or network functions, every table and column resolved through CTEs and aliases, row limit injected.

### 4. Read-only DuckDB execution

External access disabled, row- and time-bounded.

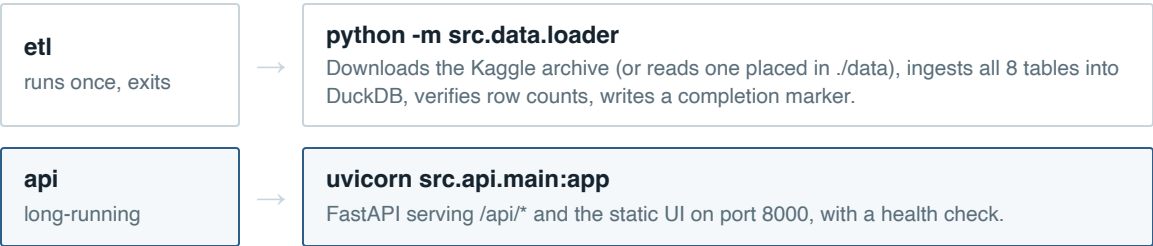
### 5. Gemini writes the answer

From the returned rows only — the summarising model never sees the database.

### Hallucination control is layered, and each layer catches a different failure.

The prompt is not a security control. A rejected query gets exactly **one** repair attempt with the specific error — a second failure is reported as a failure, not retried until something passes. The SQL is always displayed, so an answer can be checked rather than trusted.

# One image, two services, one command



To run the whole platform:

```
cp .env.example .env # add GOOGLE_API_KEY
docker compose up
```

Both services share one image. Only `etl` declares a build — giving both a build block made Compose's parallel builder race to tag the same image, which was confirmed as a real failure and is documented in the compose file itself.

**api does not wait on etl.** That is deliberate, and it is the property worth demonstrating: the trained model, the rules and the EDA artifacts are already computed and committed, so the app is useful the moment it starts.

| Capability                        | Before ETL completes |
|-----------------------------------|----------------------|
| Score a hand-entered applicant    | <div>works</div>     |
| Why this score (SHAP + narrative) | <div>works</div>     |
| Policy rules                      | <div>works</div>     |
| The portfolio (EDA)               | <div>works</div>     |
| Pick a real applicant by id       | <div>clear 503</div> |
| Ask the data                      | <div>clear 503</div> |

Once `etl` finishes, the last two start working on the next request — **no restart**. The readiness signal is a completion marker written only after row-count verification succeeds, not the existence of the database file: DuckDB creates that file the moment a connection opens, long before the tables are loaded, and a request arriving mid-build was measured surfacing as a raw 500.

# Limitations, and what would come next

---

## Known limitations

- **CODE\_GENDER is the #2 feature by global mean ISHAPI**, ahead of income, credit amount and employment history. A disparate-treatment and disparate-impact review would be required before any production use. This is a technical demonstration, not a compliance-reviewed scoring system.
- **The imbalance comparison used 4 well-known techniques at default settings.** Cost-sensitive learning, ensemble methods such as `BalancedRandomForest`, and per-fold threshold tuning were not explored, and could change the "none wins" conclusion.
- **The 10:1 false-negative-to-false-positive cost ratio is asserted, not derived** from real loan economics. It is a placeholder for whatever a deploying bank would supply — average loan size, cost of manual review, recovery rate on default.
- **Surrogate fidelity of  $R^2$  0.4885 is moderate.** The 16 rules are a useful, auditable summary — not a substitute for the model in a decision that matters.
- **No temporal validation.** The dataset carries no timestamp for out-of-time testing, so the random holdout split cannot speak to performance on genuinely future applications.

## What would come next

- Refit excluding protected attributes and measure the cost in PR-AUC, then decide explicitly whether that cost is acceptable — rather than leaving the current ranking in place by default.
- Replace the asserted cost ratio with real loan economics and re-derive the operating threshold, which moves with it.
- Extend the imbalance comparison to cost-sensitive and ensemble methods before treating "none wins" as settled.
- Source a timestamped application feed to enable out-of-time validation and drift monitoring.

**Every number in this deck is traceable.** Metrics come from `models/*.json` written by the training run; EDA figures from `models/eda_artifacts.json`; charts from `models/charts/`; screenshots from the running application against the real dataset, the real trained model and the live Gemini API. Nothing here is asserted without a corresponding artifact.

Test suite: 144 tests passing fully offline, plus 5 live API contract tests. 15 real defects were found and fixed by testing against real systems rather than synthetic fixtures.